

Homework10

Question 14.1

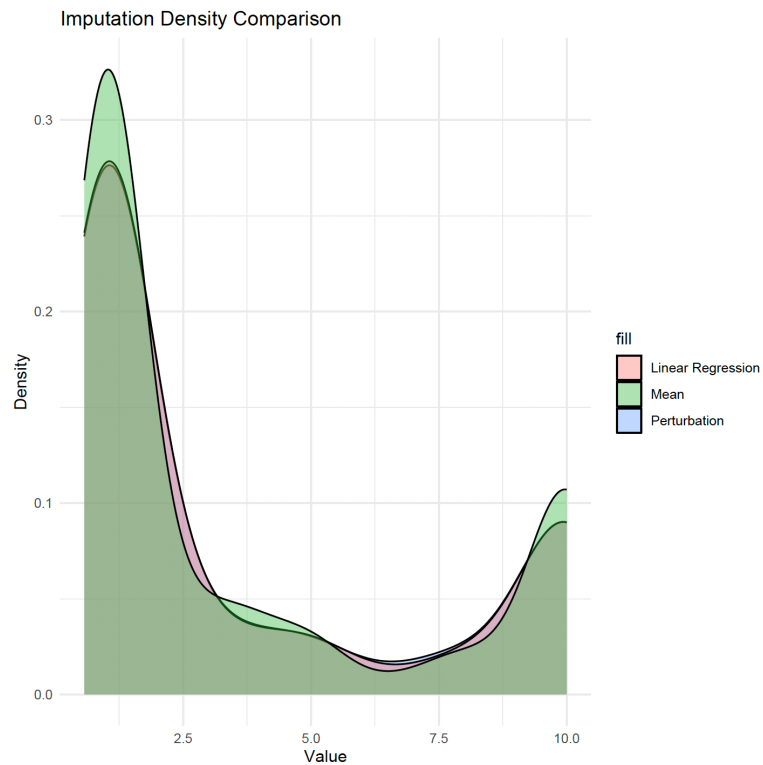
The task at hand is to address the challenge of missing data in the Wisconsin Breast Cancer datasheet, specifically focusing on the “Bare_Nuclei” attribute. The code executes a multi-stage pipeline. One of the core challenges was addressing missing entries in the Bare_Nuclei column.

Implementation:

1. **Data Partitioning:** The dataset is split into “complete” (no missing values) and “missing” subsets. The dataset was also separated into a training set (70%) and a testing set (30%) using stratified sampling via “caret::createDataPartition” to maintain the class balance between benign and malignant cases.
2. **Imputation Strategies:** Five distinct methods are applied:
 - a. **Mean Imputation:** Fills missing values with the average of the observed data. Calculated using the average of all non-missing “Bare_Nuclei” values.
 - b. **Regression Imputation:** Uses a linear model (lm) to predict missing values based on other features excluding ID and Class.
 - c. **Regression with Perturbation:** Adds random noise (Residual Standard Error) to regression predictions to restore natural variance. This was done by adding the RSE (Residual Standard Error) from the regression model to induce gaussian noise ($N(0, \sigma^2)$) back into the predicted values.
 - d. **Binary Indicator:** Mean imputation was performed, but a supplementary column Missing_Flag (0 or 1) was added to preserve the “Knowledge” of where data was originally absent.
 - e. **Dropped Rows:** A baseline approach that removes any observation with missing data.
3. **Cross-Model Validation:** Each imputed dataset is fed into both K-Nearest Neighbors (KNN) and Support Vector Machine (SVM) classifiers to evaluate how the choice of imputation affects predictive accuracy.

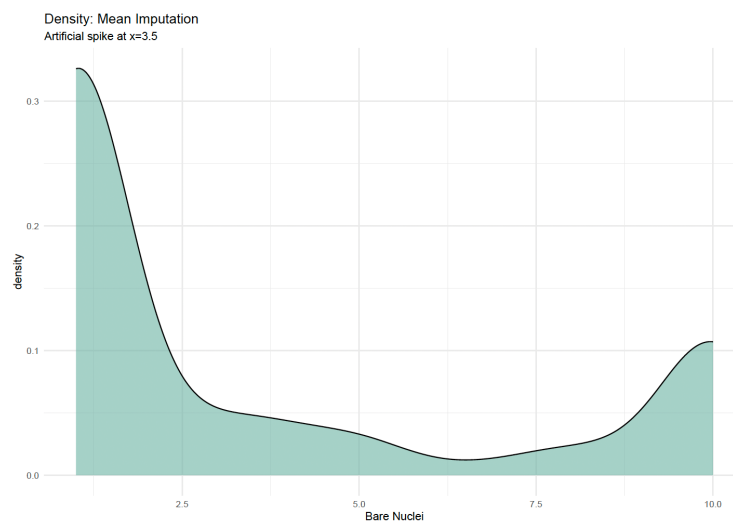
Comparison of Imputation Methods:

The choice of imputation significantly alters the statistical distribution of the features. The density plot below illustrates how each method populates the missing gaps.

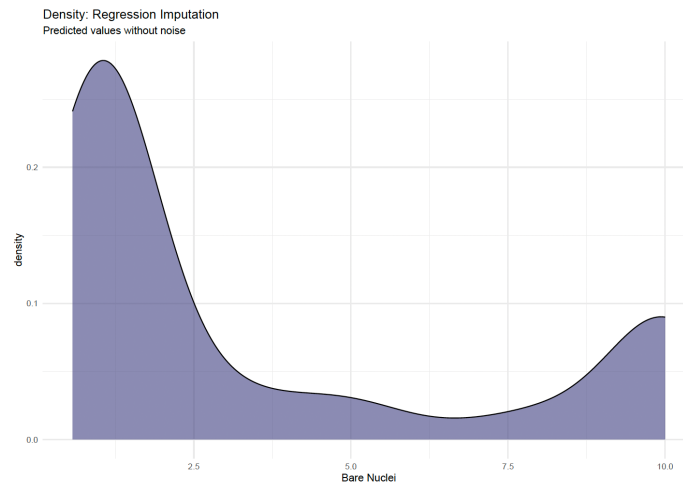


Analysis of the Density Plot:

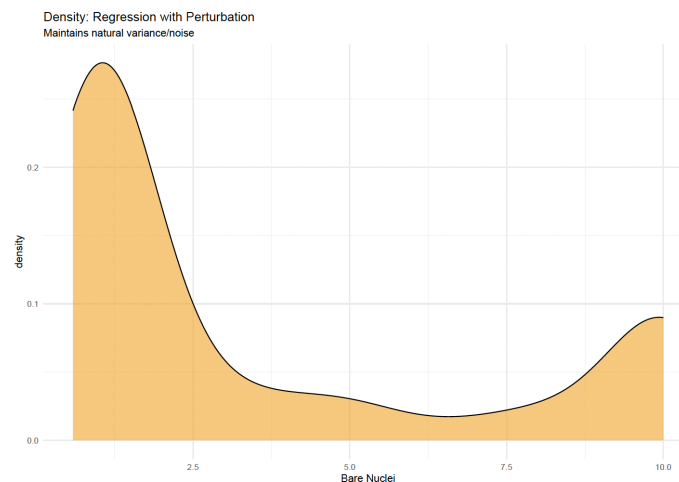
- 1. Mean Imputation (Red in the combine plot):** Creates a tall, narrow spike at the average values (~3.5). This under-represents the actual biological diversity of the samples.



- 2. Regression (Green in the combined plot):** Shifts the density toward the predicted values but still lacks the natural “spread” of the original data.



3. **Perturbation (Blue in the combined plot):** This method most closely mimics the natural distribution, maintaining the variance and preventing the model from becoming overly biased toward a single point.



Imputation Quality Metrics:

The regression model used for “Regression” and “Perturbation” methods was evaluated for its internal predictive strength:

1. **MSE (Mean Squared Error): 5.104** – Indicates that while the other features correlate with “Bare_Nuclei”, there is still significant residual variance (roughly 2.26 units on a 1-10 scale) and indicate moderate uncertainty. The regression is helpful but not a perfect mirror of the truth.
2. **AIC (3071.7) & BIC (3117.0)** – These values serve as the information-theoretic cost of the model. They confirm that while the model is complex, it provides a significantly better fit than a simple mean-only model. Also they serve as a crucial benchmark; if we removed less relevant predictors like “Mitoses” from the imputation model and

these values dropped, it would indicate a more “efficient” and potentially more accurate imputation.

Visual Comparison: The “Spike” vs. The “Spread”:

The density plot provided above reveals the primary statistical trade-off:

1. Mean Imputation create a massive, narrow spike at the average value. This artificially reduce the variance of the dataset, which can lead to “over-confident” models that fail to generalize.
2. Regression imputation narrows the distribution toward predicted trends.
3. Perturbation (shown in the plot) successfully mimics the wider, flatter distribution of the original data. This is visually the most “honest” representation of the underlying biological data.

Classification Performance with Binary Indicator:

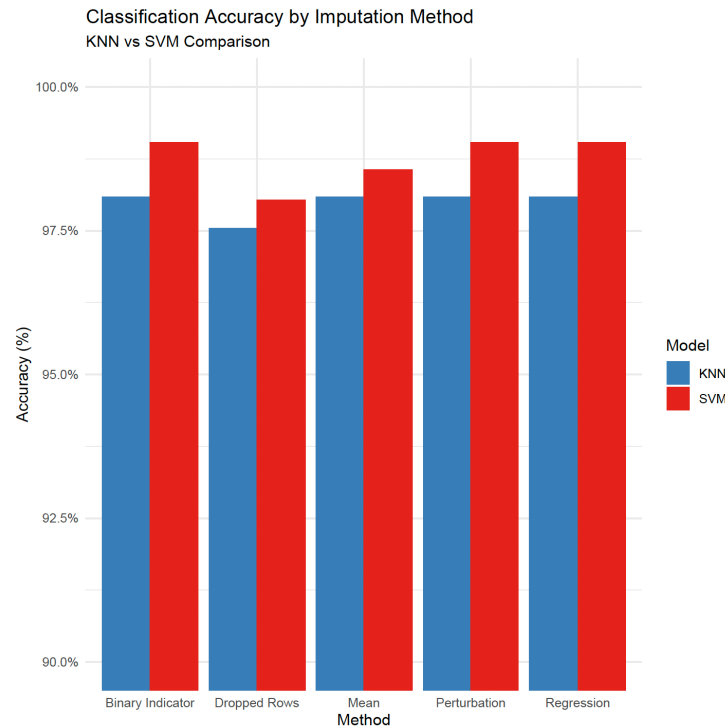
The “Binary Indicator” method achieved the top-tier accuracy of 0.9904 (with SVM).

1. **The Power of “Missingness”:** In medical datasets, data is rarely “Missing at Random” (MAR). A value might be missing because a specific test wasn’t deemed necessary by a clinician. By using the “Missing_Flag”, we allow the SVM to learn that the absence of a value is itself a predictor the class (Bening vs Malignant).
2. **Comparison to Dropping Rows:** Dropping rows resulted in the lowest accuracies (0.9755 for KNN). This confirm that even “imperfect” imputed data provides more value to a machine learning model than losing those observations entirely, as the non-missing features in those rows still contain vital diagnostic signals.

Comparison of Results/Classification Performance Analysis:

The ultimate test of an imputation method is its impact on model accuracy. The plot below compares the final classification results.

Method	KNN Accuracy	SVM Accuracy
Mean	98.09%	98.56%
Regression	98.09%	99.04%
Perturbation	98.09%	99.04%
Binary Indicator	98.09%	99.04%
Dropped Rows	97.55%	98.04%



Key Insights:

- SVM Superiority:** SVM consistently outperformed KNN across all tests. This suggests that the diagnostic boundary in this dataset is likely non-linear, which the SVM kernel handles better than the local distance-based logic of KNN.
- Binary Indicator Success:** The Binary Indicator method matched the highest accuracy (99.04%). This proves that “missingness” is often a useful feature; the model learned to use the fact that data was missing as a signal for classification.
- Baseline Feature:** “Dropped Rows” yielded the lowest accuracy. This confirms that even “imperfect” data (imputed values) is more valuable to the model than losing those observations entirely.

Conclusion:

For the Wisconsin Breast Cancer dataset, Regression with Perturbation is recommended for general statistical analysis because it preserves the data’s natural distribution. However, for maximum predictive accuracy, the Binary Indicator method paired with an SVM classifier provides the most robust diagnostic results.

Question 15.1

In chip design, a classic and critical problem for optimization is **Physical Design Closure**, specifically the **Power-Performance-Area (PPA) Trade-off**.



When designing a modern System-on-Chip (SoC), an engineer must decide how to arrange millions (or billions) of transistors and their interconnections. The goal is to maximize speed while minimizing power consumption and physical footprint. Because these goals are often in direct conflict, optimization is the only way to find an efficient solution.

The Problem: Timing-Driven Placement and Routing

As clock frequencies increase, the time available for a signal to travel from one flip-flop to the next (the "clock period") shrinks. If a signal arrives too late, the chip fails. Conversely, if you use high-power "buffer" cells to speed up the signal, the chip might overheat or the battery life will plummet.

The Optimization Challenge:

- **Objective:** Minimize the **Total Negative Slack (TNS)** (a measure of timing violations) and **Total Power**.
- **Decision Variables:** The coordinates of every logic gate, the size/strength of each transistor, and the metal layers used for routing.
- **Constraints:** The total area cannot exceed the die size, and no two wires can touch (design rule constraints).

Data Requirements

To perform this optimization effectively, you would need several categories of high-fidelity data:

1. Netlist and Connectivity Data

- a. Gate-Level Netlist:** A map of how every logic gate (AND, OR, XOR) is connected to others.
- b. Timing Constraints (SDC):** The target clock frequency (e.g., 3.0 GHz) and the required "setup" and "hold" times for the signals.

2. Technology Library Data (The "PDK")

- a. Standard Cell Library:** Data for each gate type, including its delay (in picoseconds), its leakage power, and its physical area.
- b. Wire Models:** Resistance and capacitance (RC) values for the metal layers. As wires get thinner, they become more resistive, slowing down the signal.

3. Operational Environment Data

- a. Voltage Corners:** The range of operating voltages (e.g., 0.7V to 0.9V). Chips run slower at lower voltages but save power.
- b. Thermal Profiles:** The expected heat distribution across the die. Optimization must ensure that high-activity logic isn't clustered in a way that creates "hotspots."

Applying Optimization

Optimization algorithms (often using **Simulated Annealing** or **Gradient-Based Placement**) iterate through millions of configurations.

1. **Placement:** It moves a high-speed processor core closer to its memory cache to reduce wire delay.
2. **Gate Sizing:** It swaps a large, power-hungry gate for a smaller, slower one on a "non-critical" path (where there is extra time) to save power without hurting performance.
3. **Clock Tree Synthesis:** It optimizes the delivery of the clock signal, so it reaches all parts of the chip simultaneously.

By using the above data, we can define:

1. **Decision Variables:** * $P_i = (x_i, y_i)$: The cartesian coordinates for each logic cell i .
2. **Objective Function (Minimize Wirelength and Timing Slack):** A common objective is to minimize the Half-Perimeter Wirelength (HPWL) to reduce congestion and delay:

$$\min \phi(p) = \sum_{net \in N} (\max_{i,j \in net} |x_i - x_j| + \max_{i,j \in net} |y_i - y_j|)$$

3. Constraints:

- a. **Density Constraint:** $\sum_{i \in bin} Area(i) \leq Target\ Density$ (Prevents logic cells from overlapping in a specific area of the chip).
- b. **Timing Constraint:** $T_{arrival}(k) \leq T_{required}(k)$ for all timing paths k . (signals must arrive before the clock edge.)